

GUÍA TÉCNICA · JAVA

Interfaces *funcionales* en Java



El contrato de un solo método que convirtió al comportamiento en un dato: definición formal, lambdas, referencias a métodos, la biblioteca `java.util.function`, composición y diseño de APIs profesionales.

$x \rightarrow f(x) :: 1 \text{ método}$

Java 21 LTS · compatible con Java 25

Nivel: básico → intermedio

4 laboratorios

Código compilado y verificado

CONTENIDO

- I** Fundamentos
 - 1 Por qué existen: el comportamiento como dato
 - 2 Definición formal y la anotación `@FunctionalInterface`
 - 3 Lambdas: implementar el contrato en una línea
 - 4 Referencias a métodos
 - 5 Captura de variables y closures
- II** La biblioteca estándar
 - 6 El paquete `java.util.function`
 - 7 Composición de funciones
 - 8 Especializaciones primitivas
- III** Diseño y práctica profesional
 - 9 Diseñar tus propias interfaces funcionales
 - 10 Excepciones verificadas
 - 11 Funciones de orden superior y patrones
 - 12 Trampas, buenas prácticas y el funcionamiento interno
- A-C** Laboratorios, catálogo completo y autoevaluación

OBJETIVOS DE APRENDIZAJE

- RECORDAR** la definición de interfaz funcional y las reglas de `@FunctionalInterface`.
- COMPRENDER** cómo el compilador asocia una lambda con su *tipo destino*.
- APLICAR** las interfaces de `java.util.function` y sus métodos de composición.
- ANALIZAR** errores del compilador: ambigüedad, captura, excepciones verificadas.
- EVALUAR** cuándo usar una interfaz estándar y cuándo diseñar una propia.
- CREAR** APIs pequeñas y componibles basadas en interfaces funcionales.

Requisitos previos: clases, interfaces (`implements`), métodos `default` y `static`, genéricos básicos (`List<T>`) y colecciones.

☆ CONVENCIONES DE ESTA GUÍA

Todos los **listados** marcados como **JAVA** fueron compilados y ejecutados con Java 21; la salida que aparece debajo es la salida real. Los marcados como **NO COMPILA** se compilaron a propósito para mostrar el mensaje exacto de `javac`. Las líneas **resaltadas** son las que merecen tu atención primero. Varios ejemplos comparten un modelo sencillo, el **catálogo de una tienda**, presentado en el Listado 1.

LISTADO 1 Modelo de datos compartido por los ejemplos de la guía

Producto.java

JAVA

```
1 public record Producto(String nombre, String categoria, double precio, int stock) {
2
3     public boolean disponible() {
```

```

4         return stock > 0;
5     }
6 }

```

Catalogo.java

JAVA

```

1 import java.util.List;
2
3 public final class Catalogo {
4     private Catalogo() {}
5
6     public static List<Producto> muestra() {
7         return List.of(
8             new Producto("Portátil", "tecnología", 3_200_000, 4),
9             new Producto("Mouse", "tecnología", 85_000, 25),
10            new Producto("Cuaderno", "papelería", 12_000, 0),
11            new Producto("Audífonos", "tecnología", 240_000, 8),
12            new Producto("Lápiz", "papelería", 2_500, 120),
13            new Producto("Silla ergonómica", "muebles", 890_000, 2));
14     }
15 }

```

I Fundamentos

Qué es exactamente una interfaz funcional y cómo se **implementa** con lambdas y referencias a métodos.

1 Por qué existen: el comportamiento como dato

Hasta Java 7, si querías que un método se comportara de formas distintas según la situación, tenías que escribir una **clase** para cada comportamiento. Las interfaces funcionales, junto con las lambdas (Java 8, 2014), permiten pasar un comportamiento como si fuera un valor más: guardarlo en una variable, enviarlo como argumento y devolverlo desde un método.

🗨️ ANALOGÍA

Un control remoto universal tiene un botón programable. El botón no sabe qué hará; solo sabe que, cuando lo presiones, ejecutará **una acción**. Tú decides cuál: encender el televisor, subir el volumen, abrir la puerta del garaje. Una interfaz funcional es el **botón** (el contrato de una sola acción); la lambda es la **acción** que le programas.

El mismo problema, cuatro soluciones

Supón que necesitas filtrar el catálogo con criterios cambiantes: por disponibilidad, por precio, por categoría. Observa cómo evoluciona la solución hasta llegar a la versión moderna:

LISTADO 2 Evolución: de métodos duplicados a lambdas

Main.java

JAVA

```

1 import java.util.ArrayList;
2 import java.util.List;
3
4 public class Main {
5     // Paso 1: un contrato de UNA sola operación
6     interface Criterio {
7         boolean cumple(Producto p);
8     }
9
10    // Paso 2: un único método de filtrado, parametrizado por comportamiento
11    static List<String> filtrar(List<Producto> productos, Criterio criterio) {
12        List<String> resultado = new ArrayList<>();

```

```

13     for (Producto p : productos) {
14         if (criterio.cumple(p)) resultado.add(p.nombre());
15     }
16     return resultado;
17 }
18
19 // Opción A (Java 1.0+): una clase con nombre por cada criterio
20 static class EnStock implements Criterio {
21     public boolean cumple(Producto p) { return p.disponible(); }
22 }
23
24 public static void main(String[] args) {
25     List<Producto> catalogo = Catalogo.muestra();
26
27     System.out.println("A. Clase con nombre: " + filtrar(catalogo, new EnStock()));
28
29     // Opción B (Java 1.1+): clase anónima, sin nombre pero verbosa
30     System.out.println("B. Clase anónima: " + filtrar(catalogo, new Criterio() {
31         @Override
32         public boolean cumple(Producto p) { return p.precio() < 100_000; }
33     }));
34
35     // Opción C (Java 8+): lambda
36     System.out.println("C. Lambda: " + filtrar(catalogo, p -> p.categoria().equals("muebles")));
37
38     // Opción D (Java 8+): referencia a método
39     System.out.println("D. Referencia: " + filtrar(catalogo, Producto::disponible));
40 }
41 }

```

>_ SALIDA EN CONSOLA

```

$ java Main
A. Clase con nombre: [Portátil, Mouse, Audifonos, Lápiz, Silla ergonómica]
B. Clase anónima:    [Mouse, Cuaderno, Lápiz]
C. Lambda:          [Silla ergonómica]
D. Referencia:      [Portátil, Mouse, Audifonos, Lápiz, Silla ergonómica]

```

IDEA CLAVE

Una interfaz funcional no es una característica nueva del lenguaje: es una interfaz normal con *un solo* método abstracto. Lo nuevo en Java 8 fue la sintaxis (lambdas y referencias) para implementarla sin escribir una clase.

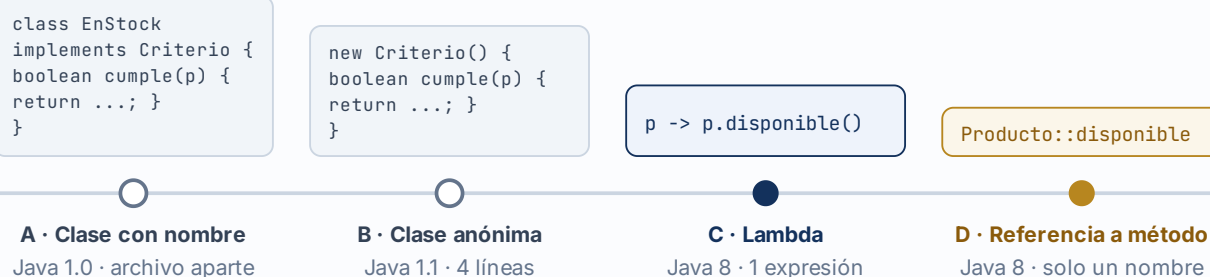


FIGURA 1 El mismo comportamiento con cuatro sintaxis. Lo que cambia es el "ruido": el contrato `Criterio` es idéntico en los cuatro casos.

2 Definición formal y `@FunctionalInterface`

DEFINICIÓN

Una **interfaz funcional** es una interfaz que tiene **exactamente un método abstracto**. A ese método se le llama **método funcional** o SAM (*Single Abstract Method*). Su firma, es decir, los tipos de sus parámetros, su tipo de retorno y las excepciones que declara, es el **descriptor funcional**: la “forma” que debe tener cualquier lambda que la implemente.

La regla dice “un método **abstracto**”, no “un método”. Por eso una interfaz funcional puede tener cualquier cantidad de métodos `default`, `static` y `private`. Hay además una excepción sutil: los métodos abstractos que coinciden con métodos públicos de `Object` (`equals`, `hashCode`, `toString`) **no cuentan**, porque toda clase ya los hereda implementados.

LISTADO 3 Una interfaz funcional válida con métodos adicionales

Main.java

JAVA

```

1  @FunctionalInterface
2  interface Transformador {
3      String aplicar(String texto);           // ← el único método abstracto (SAM)
4
5      boolean equals(Object otro);           // de Object: NO cuenta
6
7      default Transformador luego(Transformador siguiente) { // default: no cuenta
8          return texto -> siguiente.aplicar(aplicar(texto));
9      }
10
11     static Transformador identidad() {       // static: no cuenta
12         return texto -> texto;
13     }
14
15     private static String recortar(String t) { // private: no cuenta
16         return t.strip();
17     }
18
19     static Transformador limpiar() { return Transformador::recortar; }
20 }
21
22 public class Main {
23     public static void main(String[] args) {
24         Transformador gritar = texto -> texto.toUpperCase() + "!";
25         Transformador flujo = Transformador.limpiar().luego(gritar);
26         System.out.println(flujo.aplicar("  hola, lambdas  "));
27         System.out.println(Transformador.identidad().aplicar("sin cambios"));
28     }
29 }
```

>_ SALIDA EN CONSOLA

```

$ java Main
HOLA, LAMBDAS!
sin cambios
```

¿Qué hace exactamente `@FunctionalInterface` ?

La anotación es **opcional**: cualquier interfaz con un único método abstracto ya es funcional y acepta lambdas. Lo que la anotación agrega es una **verificación del compilador** y una declaración de intención para quien lea el código: “este contrato está diseñado para lambdas; no le agregues otro método abstracto”. Si alguien lo hace, la compilación falla en la propia interfaz, no en los cientos de lugares donde se usa.

Operacion.java

NO COMPILA

```

1 @FunctionalInterface
2 interface Operacion {
3     int aplicar(int a, int b);
4     int deshacer(int a, int b);    // segundo método abstracto
5 }

```

⚠ SALIDA DEL COMPILADOR

```

$ javac Operacion.java
Operacion.java:1: error: Unexpected @FunctionalInterface annotation
  Operacion is not a functional interface
    multiple non-overriding abstract methods found in interface Operacion

```

Sin la anotación, la interfaz compila, pero el error aparece más tarde y en otro lugar: cuando alguien intenta asignarle una lambda.

Main.java

NO COMPILA

```

1 interface Operacion {
2     int aplicar(int a, int b);
3     int deshacer(int a, int b);
4 }
5 public class Main {
6     public static void main(String[] args) {
7         Operacion suma = (a, b) -> a + b;
8     }
9 }

```

⚠ SALIDA DEL COMPILADOR

```

$ javac Main.java
Main.java:7: error: incompatible types: Operacion is not a functional interface
    Operacion suma = (a, b) -> a + b;
    multiple non-overriding abstract methods found in interface Operacion

```

Caso	¿Es funcional?	Motivo
Un método abstracto + varios <code>default</code> / <code>static</code>	Sí	Solo cuentan los abstractos
Un método abstracto + <code>boolean equals(Object)</code>	Sí	Los métodos públicos de <code>Object</code> no cuentan (así es <code>Comparator</code>)
Extiende una interfaz funcional sin agregar abstractos	Sí	Hereda un único SAM
Extiende dos interfaces cuyos SAM tienen la misma firma	Sí	Se combinan en un solo método
Dos métodos abstractos distintos	No	<code>multiple non-overriding abstract methods</code>
Ningún método abstracto	No	<code>no abstract method found</code>
El único método abstracto es genérico: <code><T> T aplicar(T t)</code>	Sí, pero no admite lambdas	Una lambda no puede declarar variables de tipo; solo referencias a métodos
Una clase abstracta con un solo método abstracto	No	Solo las interfaces pueden ser tipo destino de una lambda

☆ INTERFACES FUNCIONALES ANTERIORES A JAVA 8

Muchas interfaces antiguas ya tenían un solo método abstracto y, por lo tanto, aceptan lambdas sin cambios:

`Runnable` (Java 1.0), `Comparator`, `Callable`, `FileFilter`, `ActionListener`. Java 8 les añadió la anotación `@FunctionalInterface` donde correspondía. Por eso puedes escribir `new Thread(() -> System.out.println("hola"))`.

🚩 PUNTO DE CONTROL

1. Una interfaz tiene un método abstracto, dos `default` y declara `String toString()`. ¿Es funcional?
2. ¿Qué ventaja concreta aporta `@FunctionalInterface` si la interfaz ya es funcional sin ella?
3. ¿Por qué una clase abstracta con un solo método abstracto no puede recibir una lambda?

Respuestas: 1) Sí: `toString` es un método público de `Object` y no cuenta. 2) El compilador detecta en la propia interfaz cualquier cambio que rompa el contrato de un solo método. 3) Porque la especificación del lenguaje solo admite interfaces como tipo destino de una lambda.

3 Lambdas: implementar el contrato en una línea

Una **expresión lambda** es una implementación anónima del método funcional. Tiene tres partes: una lista de parámetros, la flecha `->` y un cuerpo. El cuerpo puede ser una **expresión** (su valor se devuelve automáticamente) o un **bloque** entre llaves (con `return` explícito si hay que devolver algo).

Forma	Sintaxis	Ejemplo
Sin parámetros	<code>() -> expr</code>	<code>() -> LocalDate.now()</code>
Un parámetro (paréntesis opcionales)	<code>x -> expr</code>	<code>p -> p.precio() > 0</code>
Varios parámetros	<code>(x, y) -> expr</code>	<code>(a, b) -> a + b</code>
Tipos explícitos	<code>(Tipo x, Tipo y) -> expr</code>	<code>(int a, int b) -> a * b</code>
<code>var</code> en parámetros (Java 11), útil para anotarlos	<code>(var x, var y) -> expr</code>	<code>(@NonNull var s) -> s.strip()</code>
Cuerpo de bloque	<code>(x) -> { ...; return r; }</code>	<code>s -> { var t = s.strip(); return t.isEmpty(); }</code>

El tipo destino: una lambda no tiene tipo propio

Esta es la idea más importante de la sección. La expresión `s -> s.isEmpty()`, por sí sola, **no tiene tipo**. El compilador lo deduce del **tipo destino** (*target type*): la interfaz funcional que se espera en ese lugar del código. Por eso la misma lambda puede convertirse en objetos de interfaces completamente distintas.

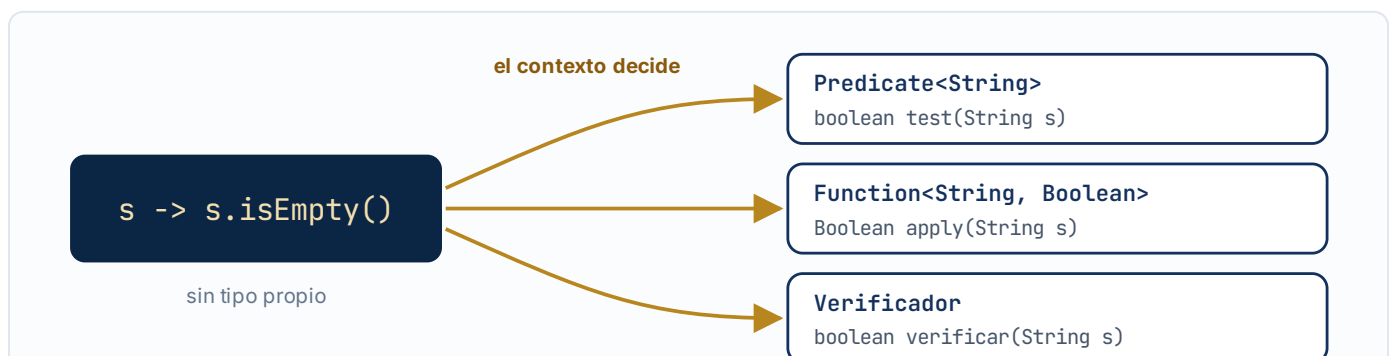


FIGURA 2 Tipado por destino. El compilador verifica que la lambda encaje en el descriptor funcional de cada interfaz: mismo número de parámetros, tipos compatibles y retorno compatible.

LISTADO 4 Una lambda, varios tipos destino

Main.java

JAVA

```

1  import java.util.ArrayList;
2  import java.util.List;
3  import java.util.concurrent.Callable;
4  import java.util.function.*;
5
6  public class Main {
7      interface Verificador { boolean verificar(String s); }
8
9      public static void main(String[] args) throws Exception {
10         Predicate<String> p = s -> s.isEmpty();
11         Function<String, Boolean> f = s -> s.isEmpty();
12         Verificador v = s -> s.isEmpty();
13         System.out.println(p.test("") + " " + f.apply("x") + " " + v.verificar(""));
14
15         // Contextos que aportan tipo destino: asignación, argumento, retorno y cast
16         Callable<String> c = () -> "desde Callable";
17         Supplier<String> su = () -> "desde Supplier";
18         System.out.println(c.call() + " | " + su.get());
19
20         Object o = (Runnable) () -> System.out.println("cast como tipo destino");
21         ((Runnable) o).run();
22
23         // Una expresión que devuelve valor también sirve donde se espera void
24         List<String> registro = new ArrayList<>();
25         Consumer<String> guardar = s -> registro.add(s); // el boolean se descarta
26         Predicate<String> guardarYConfirmar = s -> registro.add(s);
27         guardar.accept("a");
28         System.out.println(guardarYConfirmar.test("b") + " " + registro);
29     }
30 }

```

> SALIDA EN CONSOLA

```

$ java Main
true false true
desde Callable | desde Supplier
cast como tipo destino
true [a, b]

```

Si no hay tipo destino, el compilador no puede hacer nada. Es el caso de `var`, que infiere el tipo **a partir** del valor: con una lambda, ambos esperan que el otro aporte el tipo.

Main.java

NO COMPILA

```

1  public class Main {
2      public static void main(String[] args) {
3          var doble = (int x) -> x * 2;
4      }
5  }

```

⚠ SALIDA DEL COMPILADOR

```

$ javac Main.java
Main.java:3: error: cannot infer type for local variable doble
    (lambda expression needs an explicit target-type)

```

Lambda frente a clase anónima

Aunque se usan para lo mismo, no son equivalentes. La diferencia que más sorprende es el significado de `this`: dentro de una lambda se refiere al objeto que la **contiene**; dentro de una clase anónima, a la propia clase anónima.

LISTADO 5 El significado de this

Main.java

JAVA

```

1  public class Main {
2      private final String nombre = "Main";
3
4      void demostrar() {
5          Runnable anonima = new Runnable() {
6              private final String nombre = "clase anónima";    // puede tener estado propio
7              @Override
8              public void run() { System.out.println("this en anónima → " + this.nombre); }
9          };
10         Runnable lambda = () -> System.out.println("this en lambda → " + this.nombre);
11         anonima.run();
12         lambda.run();
13     }
14
15     public static void main(String[] args) {
16         new Main().demostrar();
17     }
18 }

```

> SALIDA EN CONSOLA

```

$ java Main
this en anónima → clase anónima
this en lambda → Main

```

Aspecto	Expresión lambda	Clase anónima
Qué puede implementar	Solo interfaces funcionales	Cualquier interfaz o clase abstracta, con varios métodos
Significado de <code>this</code>	La instancia que la contiene	La propia instancia anónima
Estado propio (atributos)	No	Sí
Redeclarar una variable local del método	No (already defined)	Sí (la oculta)
Archivo <code>.class</code> generado	No: se enlaza en ejecución (sección 12)	Sí: <code>Main\$1.class</code>
Cuándo preferirla	Por defecto	Cuando necesitas estado, varios métodos o una clase abstracta

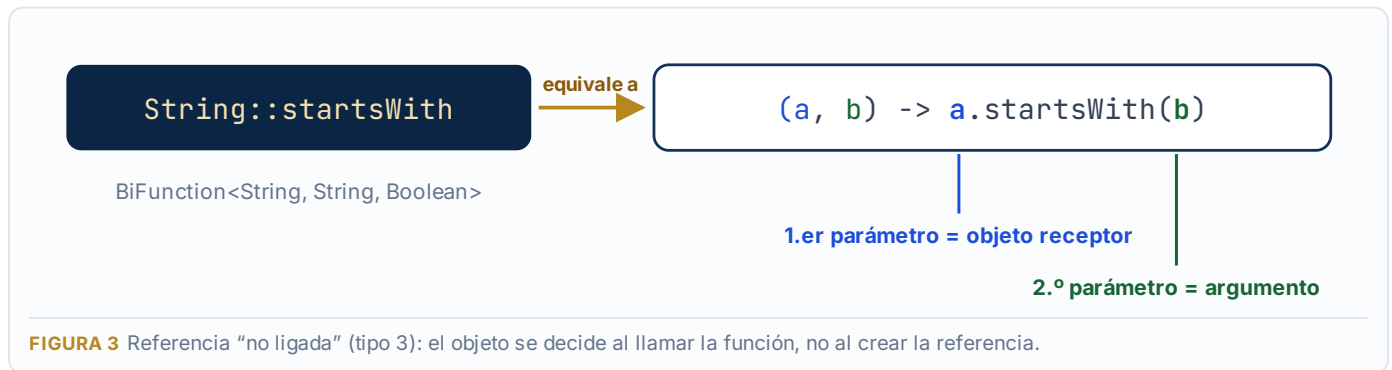
4 Referencias a métodos

Cuando una lambda solo **llama a un método que ya existe**, puedes reemplazarla por una **referencia a método** con el operador `::`. No es otro mecanismo: es una forma abreviada de escribir esa lambda, y se convierte al mismo tipo destino.

Tipo	Sintaxis	Ejemplo	Lambda equivalente
1 · Método estático	<code>Clase::estatico</code>	<code>Integer::parseInt</code>	<code>s -> Integer.parseInt(s)</code>

Tipo	Sintaxis	Ejemplo	Lambda equivalente
2 · Método de un objeto concreto	<code>objeto::metodo</code>	<code>System.out::println</code>	<code>x -> System.out.println(x)</code>
3 · Método de instancia de un objeto arbitrario	<code>Clase::metodo</code>	<code>String::toUpperCase</code>	<code>s -> s.toUpperCase()</code>
4 · Constructor	<code>Clase::new</code>	<code>ArrayList::new</code>	<code>() -> new ArrayList<>()</code>

El tipo 3 es el que más confunde. En `String::toUpperCase` no hay ningún objeto todavía: el **primer parámetro** de la función se convierte en el objeto sobre el que se invoca el método, y los parámetros restantes, en sus argumentos.



LISTADO 6 Los cuatro tipos de referencia a métodos

Main.java

JAVA

```

1  import java.util.ArrayList;
2  import java.util.List;
3  import java.util.function.*;
4
5  public class Main {
6      static double conIva(double precio) { return Math.round(precio * 1.19); }
7
8      public static void main(String[] args) {
9          Producto portatil = Catalogo.muestra().get(0);
10
11         DoubleUnaryOperator iva = Main::conIva;           // 1. estático
12         System.out.println("1 · " + iva.applyAsDouble(100_000));
13
14         String prefijo = "SKU-";
15         Function<String, String> etiquetar = prefijo::concat; // 2. objeto concreto
16         System.out.println("2 · " + etiquetar.apply("0042"));
17
18         Function<Producto, String> nombre = Producto::nombre; // 3. objeto arbitrario
19         BiFunction<String, String, Boolean> empieza = String::startsWith;
20         System.out.println("3 · " + nombre.apply(portatil) + " " + empieza.apply("Java", "Ja"));
21
22         Supplier<List<String>> nuevaLista = ArrayList::new; // 4. constructor
23         Function<String, StringBuilder> constructor = StringBuilder::new;
24         IntFunction<int[]> arreglo = int[]::new;           // también arreglos
25         List<String> lista = nuevaLista.get();
26         lista.add("ok");
27         System.out.println("4 · " + lista + " " + constructor.apply("abc").reverse()
28                             + " " + arreglo.apply(3).length);
29     }
30 }

```

>_ SALIDA EN CONSOLA

```

$ java Main
1 · 119000.0
2 · SKU-0042

```

```
3 · Portátil true
4 · [ok] cba 3
```

✓ ¿LAMBDA O REFERENCIA?

Usa la referencia cuando **solo** delega en un método con nombre claro (`Producto::nombre` , `String::isBlank`). Usa la lambda cuando hay lógica adicional, cuando reordenas argumentos o cuando la referencia sería ambigua. Una técnica profesional: si una lambda crece, extráela a un método privado con buen nombre y reemplázala por una referencia a ese método.

5 Captura de variables y closures

Una lambda puede usar variables del método donde fue creada. Se dice que las **captura**, y el resultado es una **closure**: una función que "recuerda" su entorno aunque el método que la creó ya haya terminado. La regla es estricta: solo se pueden capturar variables locales **efectivamente finales**, es decir, que nunca cambian después de asignarse (aunque no lleven la palabra `final`).

LISTADO 7 Captura de variables locales y de campos

Main.java

JAVA

```
1 import java.util.function.IntSupplier;
2 import java.util.function.Supplier;
3
4 public class Main {
5     private int contador = 0;           // campo de instancia
6
7     static Supplier<String> crearSaludo(String nombre) {
8         String saludo = "Hola, " + nombre; // efectivamente final
9         return () -> saludo + "!";        // la lambda captura su valor
10    }
11
12    IntSupplier crearContador() {
13        return () -> ++contador;           // un campo Sí puede cambiar: se captura this
14    }
15
16    public static void main(String[] args) {
17        Supplier<String> s = crearSaludo("Ana");
18        System.out.println(s.get());        // crearSaludo ya terminó
19        IntSupplier c = new Main().crearContador();
20        c.getAsInt();
21        c.getAsInt();
22        System.out.println("Contador: " + c.getAsInt());
23    }
24 }
```

>_ SALIDA EN CONSOLA

```
$ java Main
Hola, Ana!
Contador: 3
```

Main.java

NO COMPILA

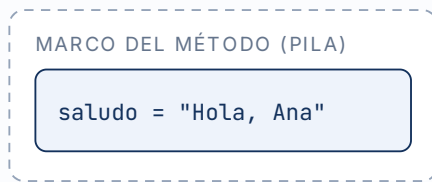
```
1 import java.util.function.Supplier;
2 public class Main {
3     public static void main(String[] args) {
4         int intentos = 0;
5         Supplier<Integer> siguiente = () -> ++intentos;
6     }
```

7 }

⚠ SALIDA DEL COMPILADOR

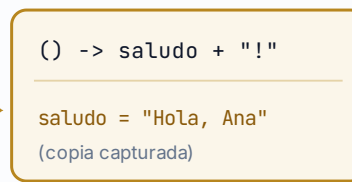
```
$ javac Main.java
Main.java:5: error: local variables referenced from a lambda expression must be final or effectively final
```

1 · Mientras corre crearSaludo("Ana")



copia

2 · Objeto lambda (montículo)



3 · El método termina

el marco se destruye...

...la lambda sigue funcionando

Se captura una **copia**: si la variable original pudiera cambiar, habría dos versiones del mismo dato.

FIGURA 4 Por qué solo se capturan variables efectivamente finales: la lambda puede vivir más que el método que la creó.

⊗ EVITA: "TRUCO" DEL ARREGLO

```
1 int[] total = {0}; // compila, pero...
2 catalogo.forEach(p -> total[0] += p.stock());
3 // estado mutable oculto y compartido:
4 // frágil con hilos y streams paralelos
```

Engaña a la regla en lugar de respetarla. Es un antipatrón frecuente en código de principiantes.

✓ PREFIERE: CALCULAR Y DEVOLVER

```
1 int total = catalogo.stream()
2     .mapToInt(Producto::stock)
3     .sum();
4 // o un for clásico, si es más claro
```

Sin estado compartido: la lambda solo **transforma**; el resultado se obtiene de forma explícita.

EN RESUMEN · PARTE I

- Una interfaz funcional tiene **un solo método abstracto**; `@FunctionalInterface` hace que el compilador lo garantice.
- Una lambda no tiene tipo propio: toma el de su **tipo destino**, y en ella `this` es el objeto que la contiene.
- Las referencias a métodos (`::`) son lambdas abreviadas de cuatro tipos; en el tipo "no ligado", el primer parámetro es el receptor.
- Las lambdas capturan **copias** de variables locales efectivamente finales; los campos sí pueden modificarse.

II La biblioteca estándar

Las interfaces que ya trae Java, cómo **combinarlas** y cómo evitar costos ocultos con tipos primitivos.

6 El paquete `java.util.function`

Antes de crear tu propia interfaz funcional, revisa este paquete: contiene **43 interfaces** que cubren casi cualquier forma de función. Todas se derivan de cuatro **familias** básicas, definidas por lo que reciben y lo que devuelven.

Interfaz	Método funcional	Descriptor	Uso típico
<code>Function<T,R></code>	<code>apply</code>	$T \rightarrow R$	Transformar o extraer: producto $\rightarrow$ nombre
<code>BiFunction<T,U,R></code>	<code>apply</code>	$(T, U) \rightarrow R$	Combinar dos datos: (producto, cantidad) $\rightarrow$ subtotal
<code>UnaryOperator<T></code>	<code>apply</code>	$T \rightarrow T$	Modificar un valor del mismo tipo
<code>BinaryOperator<T></code>	<code>apply</code>	$(T, T) \rightarrow T$	Reducir dos valores a uno: suma, mínimo
<code>Predicate<T></code>	<code>test</code>	$T \rightarrow \text{boolean}$	Filtrar y validar
<code>BiPredicate<T,U></code>	<code>test</code>	$(T, U) \rightarrow \text{boolean}$	Comparar dos datos

Interfaz	Método funcional	Descriptor	Uso típico
<code>Consumer<T></code>	<code>accept</code>	$T \rightarrow \text{void}$	Efectos: imprimir, registrar, enviar
<code>BiConsumer<T,U></code>	<code>accept</code>	$(T, U) \rightarrow \text{void}$	Efectos con dos datos: <code>map.forEach</code>
<code>Supplier<T></code>	<code>get</code>	$() \rightarrow T$	Fabricar, proveer valores o diferir un cálculo

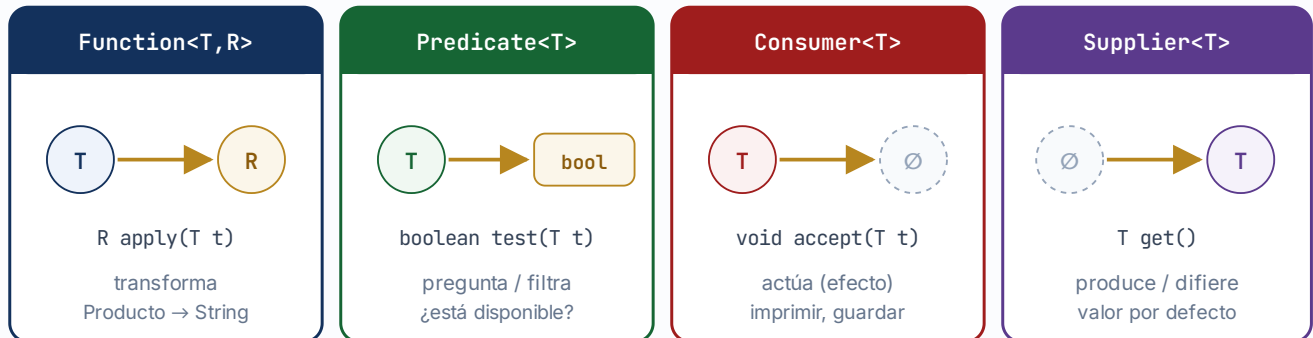


FIGURA 5 Las cuatro familias según su "forma". Las demás interfaces son variantes: dos parámetros (`Bi...`), mismo tipo de entrada y salida (`...Operator`) o tipos primitivos (`Int...`, `ToDouble...`).

LISTADO 8 Las nueve interfaces centrales aplicadas al catálogo

Main.java

JAVA

```

1  import java.util.List;
2  import java.util.function.*;
3
4  public class Main {
5      public static void main(String[] args) {
6          List<Producto> catalogo = Catalogo.muestra();
7          Producto mouse = catalogo.get(1);
8
9          Function<Producto, String> etiqueta = p -> p.nombre() + " (" + p.categoria() + ")";
10         BiFunction<Producto, Integer, Double> subtotal = (p, cantidad) -> p.precio() * cantidad;
11         UnaryOperator<String> mayus = String::toUpperCase;
12         BinaryOperator<Integer> sumar = Integer::sum;
13         Predicate<Producto> barato = p -> p.precio() < 100_000;
14         BiPredicate<Producto, String> esDe = (p, cat) -> p.categoria().equals(cat);
15         Consumer<String> imprimir = texto -> System.out.println(" · " + texto);
16         BiConsumer<String, Double> linea = (n, v) -> System.out.printf(" · %s: $%,.0f%n", n, v);
17         Supplier<Producto> porDefecto = () -> new Producto("Genérico", "otros", 0, 0);
18
19         System.out.println("Function      " + etiqueta.apply(mouse));
20         System.out.println("UnaryOperator  " + mayus.apply("lambda"));
21         System.out.println("BinaryOperator " + sumar.apply(20, 22));
22         System.out.println("Predicate    " + barato.test(mouse));
23         System.out.println("BiPredicate  " + esDe.test(mouse, "tecnología"));
24         System.out.println("Supplier     " + porDefecto.get().nombre());
25         System.out.println("Consumer y BiConsumer:");
26         imprimir.accept("sin valor de retorno");
27         linea.accept("3 × Mouse", subtotal.apply(mouse, 3));
28     }
29 }

```

> SALIDA EN CONSOLA

```

$ java Main
Function      Mouse (tecnología)
UnaryOperator LAMBDA
BinaryOperator 42

```

```

Predicate      true
BiPredicate    true
Supplier       Genérico
Consumer y BiConsumer:
  · sin valor de retorno
  · 3 × Mouse: $255.000

```

☆ ¿POR QUÉ UNARYOPERATOR SI YA EXISTE FUNCTION?

`UnaryOperator<T>` extiende `Function<T,T>` y `BinaryOperator<T>` extiende `BiFunction<T,T,T>`. No agregan métodos abstractos, así que siguen siendo funcionales. Existen para expresar la intención ("devuelve lo mismo que recibe") y para acortar los genéricos. Además, `BinaryOperator` trae las fábricas `minBy` y `maxBy`.

El paquete completo, contado por un programa

¿Son de verdad 43? En lugar de pedirte que lo creas, el siguiente programa recorre el módulo `java.base` del JDK en ejecución, lista el paquete y comprueba por reflexión cuántas interfaces llevan la anotación. El Apéndice B organiza esta misma lista por familias.

LISTADO 9 Inspeccionar java.util.function en tiempo de ejecución

Main.java

JAVA

```

1  import java.net.URI;
2  import java.nio.file.*;
3  import java.util.ArrayList;
4  import java.util.List;
5
6  public class Main {
7      public static void main(String[] args) throws Exception {
8          // El sistema de archivos "jrt:" expone las clases del propio JDK
9          FileSystem jrt = FileSystems.getFileSystem(URI.create("jrt:/"));
10         Path paquete = jrt.getPath("modules", "java.base", "java", "util", "function");
11
12         List<String> nombres = new ArrayList<>();
13         try (DirectoryStream<Path> archivos = Files.newDirectoryStream(paquete, "*.class")) {
14             for (Path archivo : archivos) {
15                 String nombre = archivo.getFileName().toString().replace(".class", "");
16                 if (!nombre.contains("$")) nombres.add(nombre); // ignora clases internas
17             }
18         }
19         nombres.sort(null);
20
21         long anotadas = 0;
22         for (String nombre : nombres) {
23             Class<?> tipo = Class.forName("java.util.function." + nombre);
24             if (tipo.isInterface() && tipo.isAnnotationPresent(FunctionalInterface.class)) anotadas++;
25         }
26         System.out.println("Tipos en java.util.function: " + nombres.size());
27         System.out.println("Interfaces con @FunctionalInterface: " + anotadas);
28         for (int i = 0; i < nombres.size(); i += 3) {
29             StringBuilder fila = new StringBuilder();
30             for (int j = i; j < Math.min(i + 3, nombres.size()); j++) {
31                 fila.append(String.format("%-22s", nombres.get(j)));
32             }
33             System.out.println(fila.toString().stripTrailing());
34         }
35     }
36 }

```

> SALIDA EN CONSOLA

```

$ java Main
Tipos en java.util.function:      43

```

```

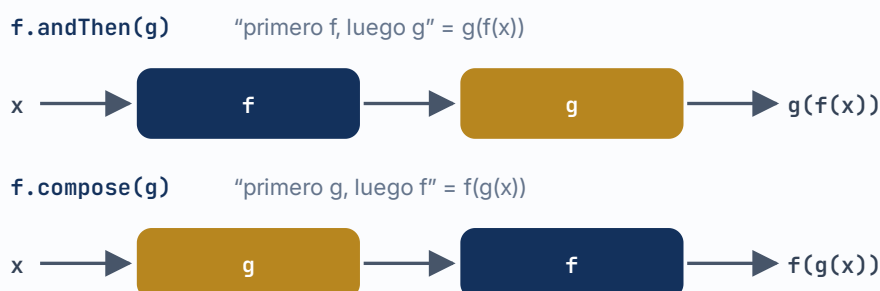
Interfaces con @FunctionalInterface: 43
BiConsumer      BiFunction      BiPredicate
BinaryOperator  BooleanSupplier Consumer
DoubleBinaryOperator DoubleConsumer DoubleFunction
DoublePredicate DoubleSupplier DoubleToIntFunction
DoubleToLongFunction DoubleUnaryOperator Function
IntBinaryOperator IntConsumer   IntFunction
IntPredicate     IntSupplier   IntToDoubleFunction
IntToLongFunction IntUnaryOperator LongBinaryOperator
LongConsumer     LongFunction   LongPredicate
LongSupplier     LongToDoubleFunction LongToIntFunction
LongUnaryOperator ObjDoubleConsumer ObjIntConsumer
ObjLongConsumer Predicate       Supplier
ToDoubleBiFunction ToDoubleFunction ToIntBiFunction
ToIntFunction     ToLongBiFunction ToLongFunction
UnaryOperator

```

7 Composición de funciones

La verdadera potencia de estas interfaces está en sus métodos `default` y `static`: permiten **componer** funciones pequeñas para construir otras más grandes, sin escribir clases nuevas. Cada método de composición **devuelve una nueva función**; las originales no cambian.

Function: `andThen`, `compose` e `identity`



Regla mnemotécnica

andThen se lee en el orden de ejecución; compose, al revés.

FIGURA 6 El orden importa cuando las operaciones no son conmutativas. En la práctica, `andThen` es mucho más legible y se usa con más frecuencia.

LISTADO 10 El orden de composición cambia el resultado

Main.java

JAVA

```

1  import java.util.List;
2  import java.util.function.Function;
3
4  public class Main {
5      public static void main(String[] args) {
6          Function<Integer, Integer> bono = precio -> precio - 10_000;    // descuento fijo
7          Function<Integer, Integer> iva = precio -> precio * 119 / 100;  // +19 %
8
9          System.out.println("IVA y luego bono: " + iva.andThen(bono).apply(100_000));
10         System.out.println("Bono y luego IVA: " + iva.compose(bono).apply(100_000));
11         System.out.println("Identidad: " + Function.<Integer>identity().apply(100_000));
12
13         // identity() es el "elemento neutro": ideal para encadenar una lista de pasos
14         List<Function<Integer, Integer>> pasos = List.of(bono, iva, bono);
15         Function<Integer, Integer> tuberia = Function.identity();
16         for (Function<Integer, Integer> paso : pasos) {
17             tuberia = tuberia.andThen(paso);
18         }
19         System.out.println("Tuberia de 3 pasos: " + tuberia.apply(100_000));

```

```
20     }
21 }
```

>_ SALIDA EN CONSOLA

```
$ java Main
IVA y luego bono: 109000
Bono y luego IVA: 107100
Identidad: 100000
Tubería de 3 pasos: 97100
```

Predicate : lógica booleana con funciones

LISTADO 11 Combinar criterios sin escribir condiciones anidadas

Main.java

JAVA

```
1  import java.util.ArrayList;
2  import java.util.List;
3  import java.util.function.Predicate;
4
5  public class Main {
6      static List<String> filtrar(List<Producto> productos, Predicate<Producto> criterio) {
7          List<String> nombres = new ArrayList<>();
8          for (Producto p : productos) if (criterio.test(p)) nombres.add(p.nombre());
9          return nombres;
10     }
11
12     public static void main(String[] args) {
13         List<Producto> catalogo = Catalogo.muestra();
14
15         Predicate<Producto> disponible = Producto::disponible;
16         Predicate<Producto> tecnologia = p -> p.categoria().equals("tecnología");
17         Predicate<Producto> caro = p -> p.precio() >= 500_000;
18
19         System.out.println("and   " + filtrar(catalogo, tecnologia.and(disponible)));
20
21         System.out.println("or    " + filtrar(catalogo, caro.or(disponible.negate())));
22
23         System.out.println("not   " + filtrar(catalogo, Predicate.not(tecnologia)));
24
25         Predicate<String> esMouse = Predicate.isEqual("Mouse");
26         System.out.println("equal  " + esMouse.test("Mouse") + " " + esMouse.test("mouse"));
27     }
28 }
```

>_ SALIDA EN CONSOLA

```
$ java Main
and   [Portátil, Mouse, Audífonos]
or    [Portátil, Cuaderno, Silla ergonómica]
not   [Cuaderno, Lápiz, Silla ergonómica]
equal true false
```

☆ NOTA TÉCNICA

`and` y `or` hacen **cortocircuito**, igual que `&&` y `||` : si el primer predicado ya decide el resultado, el segundo no se evalúa. Pon primero el criterio más barato de calcular. `Predicate.not(...)` (Java 11) es especialmente útil con referencias a métodos: `Predicate.not(String::isBlank)` .

Consumer , **BinaryOperator** y **Comparator****LISTADO 12** Encadenar efectos, elegir extremos y ordenar por varios criterios

Main.java

JAVA

```

1  import java.util.*;
2  import java.util.function.*;
3
4  public class Main {
5      public static void main(String[] args) {
6          List<Producto> catalogo = Catalogo.muestra();
7
8          List<String> bitacora = new ArrayList<>();
9          Consumer<Producto> registrar = p -> bitacora.add(p.nombre());
10         Consumer<Producto> alertar = p -> { if (p.stock() < 5) System.out.println("Stock bajo: " + p.nombre());
11     };
12         catalogo.forEach(registrar.andThen(alertar)); // ejecuta ambos, en orden
13         System.out.println("Registrados: " + bitacora.size());
14
15         Comparator<Producto> porPrecio = Comparator.comparingDouble(Producto::precio);
16         BinaryOperator<Producto> masBarato = BinaryOperator.minBy(porPrecio);
17         BinaryOperator<Producto> masCaro = BinaryOperator.maxBy(porPrecio);
18         Producto min = catalogo.get(0), max = catalogo.get(0);
19         for (Producto p : catalogo) { min = masBarato.apply(min, p); max = masCaro.apply(max, p); }
20         System.out.println("Extremos: " + min.nombre() + " / " + max.nombre());
21
22         Comparator<Producto> orden = Comparator.comparing(Producto::categoria)
23             .thenComparing(Producto::precio, Comparator.reverseOrder());
24         List<Producto> copia = new ArrayList<>(catalogo);
25         copia.sort(orden);
26         copia.forEach(p -> System.out.printf(" %-11s %s\n", p.categoria(), p.nombre()));
27     }
28 }

```

>_ SALIDA EN CONSOLA

```

$ java Main
Stock bajo: Portátil
Stock bajo: Cuaderno
Stock bajo: Silla ergonómica
Registrados: 6
Extremos: Lápiz / Portátil
muebles      Silla ergonómica
papelería    Cuaderno
papelería    Lápiz
tecnología   Portátil
tecnología   Audífonos
tecnología   Mouse

```

Interfaz	Métodos de composición	Tipo
<code>Function</code>	<code>andThen</code> , <code>compose</code> , <code>identity</code>	default, default, static
<code>BiFunction</code>	<code>andThen(Function)</code>	default
<code>Predicate</code>	<code>and</code> , <code>or</code> , <code>negate</code> , <code>not</code> (Java 11), <code>isEqual</code>	default ×3, static ×2
<code>Consumer</code> / <code>BiConsumer</code>	<code>andThen</code>	default
<code>UnaryOperator</code>	<code>identity</code>	static
<code>BinaryOperator</code>	<code>minBy</code> , <code>maxBy</code>	static
<code>Comparator</code>	<code>comparing</code> , <code>thenComparing</code> , <code>reversed</code> , <code>nullsFirst</code> ...	static y default

Laboratorio 1 · Generador de *slugs* por composición

BÁSICO · 30 MIN

Objetivo: construir una transformación compleja componiendo funciones pequeñas. **Secciones:** 4 · 6 · 7

Una tienda en línea necesita convertir nombres de productos en *slugs* para sus URL: "Silla Ergonómica" → silla-ergonomica.

- 1 **Declara cinco pasos** como constantes `UnaryOperator<String>`: recortar espacios (referencia a método), pasar a minúsculas, quitar tildes, eliminar todo lo que no sea letra, dígito, espacio o guion, y reemplazar espacios y guiones repetidos por un único guion.
- 2 **Para quitar tildes**, usa `Normalizer.normalize(s, Normalizer.Form.NFD)` y elimina las marcas con `replaceAll("\\p{M}", "")`.
- 3 **Escribe** `tuberia(List<? extends Function<String, String>> pasos)`, que parta de `Function.identity()` y encadene cada paso con `andThen`.
- 4 **Prueba** la tubería con los tres nombres de la salida esperada. Luego cambia el orden de dos pasos y explica por qué el resultado cambia (o no).

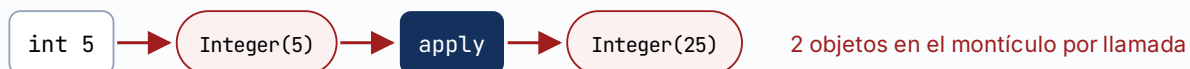
> SALIDA EN CONSOLA

```
$ java Main
" Silla Ergonómica "      → silla-ergonomica
"Audifonos Bluetooth 5.3" → audifonos-bluetooth-53
"!Oferta! Portátil i7 -- 16GB" → oferta-portatil-i7-16gb
```

8 Especializaciones primitivas

Los genéricos de Java solo trabajan con objetos: `Function<Integer, Integer>` no puede recibir un `int`, así que el valor se "envuelve" en un objeto `Integer` (**autoboxing**) y se "desenvuelve" al usarlo. En un cálculo aislado no importa; dentro de un ciclo de millones de iteraciones, crea millones de objetos temporales. Para evitarlo existen **versiones primitivas** de las interfaces.

`Function<Integer, Integer>`



`IntUnaryOperator`



FIGURA 7 El costo oculto del autoboxing. Las interfaces primitivas usan métodos con nombre propio (`applyAsInt`, `test`, `getAsDouble`) para no chocar con las versiones genéricas.

Patrón de nombre	Significado	Ejemplo	Descriptor
<code>Int...</code> , <code>Long...</code> , <code>Double...</code>	La entrada es primitiva	<code>IntPredicate</code>	<code>int → boolean</code>
<code>To{Int Long Double}...</code>	La salida es primitiva	<code>ToDoubleFunction<T></code>	<code>T → double</code>
<code>{Int}To{Long}...</code>	Entrada y salida primitivas distintas	<code>IntToDoubleFunction</code>	<code>int → double</code>
<code>...UnaryOperator</code> / <code>...BinaryOperator</code>	Operación del mismo tipo primitivo	<code>IntBinaryOperator</code>	<code>(int, int) → int</code>
<code>Obj{Int}Consumer<T></code>	Un objeto y un primitivo	<code>ObjIntConsumer<T></code>	<code>(T, int) → void</code>
<code>{Int}Supplier</code> , <code>BooleanSupplier</code>	Produce un primitivo	<code>DoubleSupplier</code>	<code>() → double</code>

LISTADO 13 Interfaces primitivas en acción

Main.java

JAVA

```

1  import java.util.List;
2  import java.util.function.*;
3
4  public class Main {
5      public static void main(String[] args) {
6          List<Producto> catalogo = Catalogo.muestra();
7          ToDoubleFunction<Producto> valorEnInventario = p -> p.precio() * p.stock();
8          IntPredicate esPar = n -> n % 2 == 0;
9          IntUnaryOperator cuadrado = n -> n * n;
10         IntBinaryOperator maximo = Math::max;
11         ObjIntConsumer<String> repetir = (texto, veces) -> System.out.println(texto.repeat(veces));
12         IntFunction<String> barra = "█"::repeat;
13
14         double total = 0;
15         for (Producto p : catalogo) total += valorEnInventario.applyAsDouble(p);
16         System.out.printf("Valor del inventario: $%,.0f%n", total);
17
18         System.out.println(esPar.test(42) + " " + cuadrado.applyAsInt(12) + " " + maximo.applyAsInt(7, 3));
19         System.out.println(cuadrado.andThen(n -> n + 1).applyAsInt(5)); // también componen
20         repetir.accept("=-", 8);
21         for (Producto p : catalogo) {
22             System.out.printf("%-17s %s%n", p.nombre(), barra.apply(p.stock() / 10));
23         }
24     }
25 }

```

> SALIDA EN CONSOLA

```

$ java Main
Valor del inventario: $18.925.000
true 144 7
26
=====
Portátil
Mouse      █
Cuaderno
Audífonos
Lápiz      ████████
Silla ergonómica

```

⚠ ATENCIÓN

No hay especializaciones para todo: no existen `BooleanFunction`, `CharPredicate` ni `FloatUnaryOperator`. El JDK solo cubre `int`, `long` y `double` (más `BooleanSupplier`). Para `char` o `short`, usa la versión `int`: el valor se amplía sin crear objetos.

EN RESUMEN · PARTE II

- `java.util.function` tiene 43 interfaces organizadas en cuatro familias: **Function**, **Predicate**, **Consumer** y **Supplier**.
- Sus métodos `default` y `static` permiten **componer** comportamiento: `andThen`, `compose`, `and`, `or`, `negate`, `minBy`.
- Las versiones primitivas (`IntPredicate`, `ToDoubleFunction` ...) evitan el costo del autoboxing en código intensivo.

III**Diseño y práctica profesional**

Cómo diseñar tus propias interfaces, manejar excepciones y aplicar **patrones** reales.

9 Diseñar tus propias interfaces funcionales

La regla general es **reutilizar** `java.util.function`: otros programadores ya conocen esas interfaces y sus métodos de composición. Pero hay situaciones en las que una interfaz propia comunica mejor o resuelve algo que el JDK no cubre.

Situación	Decisión	Ejemplo
La forma coincide con una interfaz estándar y el significado es genérico	Estándar	<code>Predicate<Producto></code> para filtrar
El concepto tiene nombre propio en tu dominio	Propia	<code>TarifaEnvio</code> es más claro que <code>BiFunction<Double, Integer, Double></code>
Necesitas 3 o más parámetros	Propia	El JDK no trae <code>TriFunction</code>
El método debe lanzar excepciones verificadas	Propia	<code>R aplicar(T t) throws IOException</code> (sección 10)
Quieres métodos de composición propios del dominio	Propia	<code>tarifa.conRecargo(50).conMinimo(15_000)</code>

LISTADO 14 Una interfaz genérica de tres parámetros y una interfaz de dominio componible

Main.java

JAVA

```

1  import java.util.Objects;
2  import java.util.function.Function;
3
4  @FunctionalInterface
5  interface TriFunction<A, B, C, R> {
6      R aplicar(A a, B b, C c);
7
8      default <V> TriFunction<A, B, C, V> luego(Function<? super R, ? extends V> despues) {
9          Objects.requireNonNull(despues); // falla pronto y con mensaje claro
10         return (a, b, c) -> despues.apply(aplicar(a, b, c));
11     }
12 }
13
14 @FunctionalInterface
15 interface TarifaEnvio {
16     double calcular(double pesoKg, int distanciaKm);
17
18     default TarifaEnvio conRecargo(double porcentaje) {
19         return (peso, km) -> calcular(peso, km) * (1 + porcentaje / 100);
20     }
21
22     default TarifaEnvio conMinimo(double minimo) {
23         return (peso, km) -> Math.max(minimo, calcular(peso, km));
24     }
25
26     static TarifaEnvio porPesoYDistancia(double porKg, double porKm) {
27         return (peso, km) -> peso * porKg + km * porKm;
28     }
29 }
30
31 public class Main {
32     public static void main(String[] args) {
33         TriFunction<String, Integer, Double, String> factura =
34             (cliente, unidades, precio) -> cliente + " debe " + unidades * precio;
35         System.out.println(factura.aplicar("Ana", 3, 2.5));
36         System.out.println(factura.luego(String::length).aplicar("Ana", 3, 2.5) + " caracteres");
37
38         TarifaEnvio estandar = TarifaEnvio.porPesoYDistancia(2_000, 150);

```

```

39     TarifaEnvio urgente = estandar.conRecargo(50).conMinimo(15_000);
40     System.out.println(" kg km estándar urgente");
41     double[][] envios = { {1.5, 10}, {0.2, 5}, {8, 120} };
42     for (double[] e : envios) {
43         System.out.printf("%5.1f %5.0f $%,9.0f $%,9.0f%n", e[0], e[1],
44             estandar.calcular(e[0], (int) e[1]), urgente.calcular(e[0], (int) e[1]));
45     }
46 }
47 }

```

> SALIDA EN CONSOLA

```

$ java Main
Ana debe 7.5
12 caracteres
 kg km estándar urgente
1,5 10 $ 4.500 $ 15.000
0,2 5 $ 1.150 $ 15.000
8,0 120 $ 34.000 $ 51.000

```

❏ POR QUÉ ANDTHEN RECIBE FUNCTION<? SUPER R, ? EXTENDS V>

Los comodines hacen que la composición sea **flexible**. `? super R`: la función siguiente puede aceptar un tipo más general que `R` (una `Function<Object, ...>` sirve después de una que produce `String`). `? extends V`: puede producir un subtipo del resultado declarado. Es la regla **PECS** (*Producer Extends, Consumer Super*): lo que la función **consume** usa `super`; lo que **produce**, `extends`. Copia este patrón en tus propios métodos de composición.

✔ LISTA DE VERIFICACIÓN PARA UNA INTERFAZ FUNCIONAL PROPIA

1) Anótala con `@FunctionalInterface`. 2) Dale un nombre de **rol** (`TarifaEnvio`, `Regla`, `Formateador`), no de implementación. 3) Nombra el método con un verbo claro. 4) Agrega métodos `default` de composición que devuelvan **el mismo tipo**, para poder encadenarlos. 5) Valida con `Objects.requireNonNull` los argumentos de esos métodos. 6) Ofrece fábricas `static` para los casos comunes.

🧪 Laboratorio 2 · Motor de validación componible

INTERMEDIO · 45 MIN

Objetivo: diseñar una interfaz funcional genérica con composición y fábricas. **Secciones:** 2 · 7 · 9

Vas a crear una pequeña API de validación para formularios, similar en espíritu a las bibliotecas profesionales. Al final, validar un registro debe verse así: `usuarioValido.y(correoValido).y(edadValida)`.

- 1) **Declara** `@FunctionalInterface interface Regla<T>` con el método `List<String> validar(T valor)`: lista vacía si es válido; con mensajes, si no.
- 2) **Agrega** `default Regla<T> y(Regla<? super T> otra)`, que ejecute ambas reglas y una sus errores.
- 3) **Agrega** la fábrica `static <T> Regla<T> exigir(Predicate<? super T> condicion, String mensaje)`.
- 4) **Agrega** la fábrica `static <T, C> Regla<T> campo(Function<? super T, ? extends C> extractor, Regla<? super C> regla)` que aplica una regla sobre un campo del objeto.
- 5) **Crea** record `Registro(String usuario, String correo, int edad)` y tres reglas: usuario obligatorio y de 4 o más caracteres, correo con formato válido y edad mínima de 14 años.
- 6) **Valida** los tres registros de la solución y compara tu salida:

> SALIDA EN CONSOLA

```

$ java Main
VÁLIDO   camila_r   []
INVÁLIDO (vacío)   [usuario: obligatorio, usuario: mínimo 4 caracteres, correo: formato inválido, edad:

```

```

mínimo 14 años]
INVÁLIDO ana      [usuario: mínimo 4 caracteres, correo: formato inválido]

```

10 Excepciones verificadas

Ninguna interfaz de `java.util.function` declara `throws`. Por lo tanto, una lambda que implemente `Function` **no puede** dejar escapar una excepción verificada (*checked*) como `IOException` o `URISyntaxException`. El compilador lo detecta de inmediato:

Main.java NO COMPILA

```

1 import java.net.URI;
2 import java.util.function.Function;
3 public class Main {
4     public static void main(String[] args) {
5         Function<String, URI> aUri = texto -> new URI(texto);
6     }
7 }

```

⚠ SALIDA DEL COMPILADOR

```

$ javac Main.java
Main.java:5: error: unreported exception URISyntaxException; must be caught or declared to be thrown

```

Hay tres soluciones, de la menos a la más reutilizable:

1. **try/catch dentro de la lambda.** Funciona, pero convierte una línea en seis y se repite en cada uso.
2. **Usar una interfaz del JDK que sí declare `throws`,** como `Callable<V>` (`V call() throws Exception`), cuando el contexto lo permita.
3. **Crear una interfaz funcional propia con `throws` y un adaptador** que la convierta en una `Function` estándar. Es la técnica profesional y la que usan bibliotecas como Vavr o Apache Commons.

LISTADO 15 Interfaz con throws genérico y adaptador a Function

Main.java JAVA

```

1 import java.net.URI;
2 import java.util.List;
3 import java.util.function.Function;
4
5 public class Main {
6
7     @FunctionalInterface
8     interface FuncionConExcepcion<T, R, E extends Exception> {
9         R aplicar(T t) throws E;           // el tipo de excepción también es genérico
10    }
11
12    static <T, R> Function<T, R> sinVerificar(FuncionConExcepcion<T, R, ?> funcion) {
13        return entrada -> {
14            try {
15                return funcion.aplicar(entrada);
16            } catch (RuntimeException e) {
17                throw e;           // las no verificadas pasan intactas
18            } catch (Exception e) {
19                throw new IllegalArgumentException("'" + entrada + "' -> " + e.getMessage(), e);
20            }
21        };
22    }
23
24    public static void main(String[] args) {

```

```

25     Function<String, URI> aUri = sinVerificar(URI::new); // ¡ahora sí es una Function!
26
27     for (String texto : List.of("https://tienda.co/productos?id=42", "https://tienda .co")) {
28         try {
29             System.out.println("OK     host = " + aUri.apply(texto).getHost());
30         } catch (IllegalArgumentException e) {
31             System.out.println("ERROR  " + e.getMessage());
32         }
33     }
34 }
35 }

```

>_ SALIDA EN CONSOLA

```

$ java Main
OK     host = tienda.co
ERROR  'https://tienda .co' → Illegal character in authority at index 14: https://tienda .co

```

⚠ NO OCULTES EL ERROR

Envolver una excepción verificada en una no verificada es legítimo solo si **conservas la causa** (el segundo argumento del constructor, `e`) y el mensaje identifica el dato que falló. Nunca hagas `catch (Exception e) { return null; }` dentro de una lambda: convierte un error claro en un `NullPointerException` misterioso varias líneas después.

11 Funciones de orden superior y patrones

Una **función de orden superior** es un método que **recibe** una función, la **devuelve**, o ambas cosas. Con interfaces funcionales, muchos patrones de diseño clásicos se reducen a unas pocas líneas.

Fábricas de funciones y currificación

Un método que devuelve una lambda configura comportamiento una vez y lo reutiliza muchas. Llevado al extremo, una función de varios parámetros puede expresarse como una cadena de funciones de un parámetro: eso se llama **currificación**.

LISTADO 16 Métodos que devuelven funciones

Main.java

JAVA

```

1  import java.util.ArrayList;
2  import java.util.List;
3  import java.util.function.*;
4
5  public class Main {
6      // Fábrica: cada llamada configura y devuelve un criterio nuevo
7      static Predicate<Producto> precioEntre(double minimo, double maximo) {
8          return p -> p.precio() >= minimo && p.precio() <= maximo;
9      }
10
11     static DoubleUnaryOperator descuento(double porcentaje) {
12         return precio -> precio * (1 - porcentaje / 100);
13     }
14
15     public static void main(String[] args) {
16         List<String> rango = new ArrayList<>();
17         for (Producto p : Catalogo.muestra()) if (precioEntre(50_000, 500_000).test(p)) rango.add(p.nombre());
18         System.out.println("Entre $50.000 y $500.000: " + rango);
19
20         // Currificación: porcentaje -> (precio -> resultado)
21         DoubleFunction<DoubleUnaryOperator> descuentoCurry = pct -> precio -> precio * (1 - pct / 100);

```

```

22     DoubleUnaryOperator blackFriday = descuentoCurry.apply(40);
23     System.out.println(descuento(15).applyAsDouble(200_000) + " | " + blackFriday.applyAsDouble(200_000));
24 }
25 }

```

>_ SALIDA EN CONSOLA

```

$ java Main
Entre $50.000 y $500.000: [Mouse, Audífonos]
170000.0 | 120000.0

```

Evaluación diferida con `Supplier`

Un `Supplier` representa un valor **que todavía no se ha calculado**. Si un método recibe el valor directamente, el cálculo se hace siempre, aunque no se use; si recibe un `Supplier`, puede decidir calcularlo solo cuando haga falta.

LISTADO 17 Calcular solo cuando es necesario

Main.java

JAVA

```

1  import java.util.function.Supplier;
2
3  public class Main {
4      static int evaluaciones = 0;
5
6      static String reporteCostoso() {
7          evaluaciones++; // simula una consulta cara
8          return "inventario de " + Catalogo.muestra().size() + " productos";
9      }
10
11     static void registrar(boolean activo, String mensaje) {
12         if (activo) System.out.println("[LOG] " + mensaje);
13     }
14
15     static void registrarDiferido(boolean activo, Supplier<String> mensaje) {
16         if (activo) System.out.println("[LOG] " + mensaje.get()); // solo aquí se calcula
17     }
18
19     public static void main(String[] args) {
20         registrar(false, reporteCostoso()); // se calcula... y se descarta
21         registrarDiferido(false, Main::reporteCostoso); // no se calcula
22         registrarDiferido(true, Main::reporteCostoso); // se calcula porque se usa
23         System.out.println("Evaluaciones: " + evaluaciones);
24     }
25 }

```

>_ SALIDA EN CONSOLA

```

$ java Main
[LOG] inventario de 6 productos
Evaluaciones: 2

```

☆ EL JDK USA ESTE PATRÓN

`Logger.fine(Supplier<String>)`, `Objects.requireNonNull(obj, Supplier<String>)` y `Optional.orElseGet(Supplier)` reciben un `Supplier` precisamente para no construir mensajes o valores que quizá nunca se usen.

Memoización: recordar resultados

LISTADO 18 Envolver cualquier función con una caché

Main.java

JAVA

```

1  import java.util.HashMap;

```

```

2  import java.util.List;
3  import java.util.Map;
4  import java.util.function.Function;
5
6  public class Main {
7      static <T, R> Function<T, R> memorizar(Function<T, R> funcion) {
8          Map<T, R> cache = new HashMap<>();
9          return entrada -> cache.computeIfAbsent(entrada, funcion);
10     }
11
12     static int calculosReales = 0;
13
14     static long tarifaPorCiudad(String ciudad) {           // imagina una consulta lenta
15         calculosReales++;
16         return ciudad.length() * 1_000L;
17     }
18
19     public static void main(String[] args) {
20         Function<String, Long> tarifa = memorizar(Main::tarifaPorCiudad);
21         List<String> consultas = List.of("Bogotá", "Cali", "Bogotá", "Medellín", "Cali", "Bogotá");
22         for (String ciudad : consultas) System.out.print(ciudad + "=" + tarifa.apply(ciudad) + " ");
23         System.out.println();
24         System.out.println("Cálculos reales: " + calculosReales + " de " + consultas.size() + " consultas");
25     }
26 }

```

> SALIDA EN CONSOLA

```

$ java Main
Bogotá=6000 Cali=4000 Bogotá=6000 Medellín=8000 Cali=4000 Bogotá=6000
Cálculos reales: 3 de 6 consultas

```

⚠ ATENCIÓN

Esta caché usa un `HashMap`: no es segura si varios hilos la usan a la vez (usa `ConcurrentHashMap`), y la función memorizada **no debe** llamarse a sí misma a través de la caché, porque `computeIfAbsent` prohíbe modificar el mapa durante el cálculo.

Comando y observador

LISTADO 19 Tareas diferidas con Runnable y suscriptores con Consumer

Main.java

JAVA

```

1  import java.util.ArrayList;
2  import java.util.List;
3  import java.util.function.Consumer;
4
5  public class Main {
6      public static void main(String[] args) {
7          List<Consumer<String>> suscriptores = new ArrayList<>();           // observador
8          suscriptores.add(evento -> System.out.println(" [correo] " + evento));
9          suscriptores.add(evento -> System.out.println(" [app] " + evento));
10         Consumer<String> publicar = evento -> suscriptores.forEach(s -> s.accept(evento));
11
12         List<Runnable> cola = new ArrayList<>();                           // comando
13         cola.add(() -> publicar.accept("Pedido 101 recibido"));
14         cola.add(() -> publicar.accept("Pedido 101 despachado"));
15         System.out.println("Tareas en cola: " + cola.size() + " (aún no se ejecutan)");
16         cola.forEach(Runnable::run);
17     }
18 }

```

>_ SALIDA EN CONSOLA

```
$ java Main
Tareas en cola: 2 (aún no se ejecutan)
[correo] Pedido 101 recibido
[app]    Pedido 101 recibido
[correo] Pedido 101 despachado
[app]    Pedido 101 despachado
```

Patrón clásico	Antes (clases)	Con interfaces funcionales
Estrategia	Una interfaz y una clase por algoritmo	Parámetro <code>Function</code> / <code>Predicate</code> o un <code>Map</code> de lambdas (Laboratorio 3)
Comando	Clase <code>Comando</code> con <code>ejecutar()</code>	<code>Runnable</code> o <code>Consumer<T></code> en una cola
Observador	Interfaz <code>Listener</code> + clases	<code>List<Consumer<Evento>></code>
Fábrica	Clase <code>Factory</code>	<code>Supplier<T></code> o <code>Clase::new</code>
Decorador	Clases que envuelven a otras	<code>andThen</code> , <code>compose</code> o métodos <code>default</code> como <code>conRecargo</code>
Método plantilla	Clase abstracta con "ganchos"	Un método que recibe los pasos variables como funciones

 Laboratorio 3 · Calculadora con estrategias

BÁSICO · 30 MIN

Objetivo: reemplazar un `switch` por un mapa de estrategias. **Secciones:** 4 · 8 · 11

- 1 **Crea** un `Map<String, DoubleBinaryOperator>` (usa `LinkedHashMap` para conservar el orden) con los operadores `+` `-` `*` `/` `^` `max`. Usa referencias a métodos donde sea posible: `Double::sum`, `Math::pow`, `Math::max`.
- 2 **La división** debe lanzar `ArithmeticException("división por cero")` si el divisor es 0.
- 3 **Escribe** `double evaluar(String expresion)` para textos como `"12 + 30"`: separa con `split("\\s+")`, busca la estrategia y aplícala. Lanza `IllegalArgumentException` si el formato o el operador no son válidos.
- 4 **Evalúa** las seis expresiones de la salida esperada, atrapando los errores.
- 5 **Reflexiona:** ¿qué tendrías que cambiar para agregar el operador `%`? Compáralo con una versión basada en `switch`.

>_ SALIDA EN CONSOLA

```
$ java Main
12 + 30 = 42.0
2 ^ 10  = 1024.0
7 / 2   = 3.5
9 max 4 = 9.0
5 / 0   → error: división por cero
3 % 2   → error: operador desconocido: %
Operadores disponibles: [+ , - , * , / , ^ , max]
```

12 Trampas, buenas prácticas y funcionamiento interno

Trampa 1 · Sobrecarga ambigua

Si un método está sobrecargado con dos interfaces funcionales de forma compatible, el compilador no puede elegir el tipo destino de la lambda:

Main.java

NO COMPILA

```

1 import java.util.concurrent.Callable;
2 import java.util.function.Supplier;
3 public class Main {
4     static void ejecutar(Supplier<String> tarea) { System.out.println("Supplier"); }
5     static void ejecutar(Callable<String> tarea) { System.out.println("Callable"); }
6     public static void main(String[] args) {
7         ejecutar(() -> "hola");
8     }
9 }

```

⚠ SALIDA DEL COMPILADOR

```

$ javac Main.java
Main.java:7: error: reference to ejecutar is ambiguous
    ejecutar(() -> "hola");
    ^
    both method ejecutar(Supplier<String>) in Main and method ejecutar(Callable<String>) in Main match

```

Solución de diseño: no sobrecargues métodos con interfaces funcionales de la misma aridad; usa nombres distintos (`ejecutar` y `ejecutarConExcepcion`). Como último recurso, quien llama puede desambiguar con un cast: `ejecutar((Supplier<String>) () -> "hola")` .

Trampa 2 · Métodos funcionales genéricos

Si el propio método abstracto declara una variable de tipo, la interfaz sigue siendo funcional, pero ninguna lambda puede implementarla (una lambda no tiene dónde declarar `<T>`). Solo sirven referencias a métodos genéricos. Pon el parámetro de tipo en la **interfaz**, no en el método.

Main.java

NO COMPILA

```

1 interface Identidad {
2     <T> T aplicar(T valor);
3 }
4 public class Main {
5     public static void main(String[] args) {
6         Identidad id = valor -> valor;
7     }
8 }

```

⚠ SALIDA DEL COMPILADOR

```

$ javac Main.java
Main.java:6: error: incompatible types: invalid functional descriptor for lambda expression
    method <T>(T)T in interface Identidad is generic

```

Trampa 3 · Efectos secundarios ocultos

⊗ EVITA

```

1 List<String> agotados = new ArrayList<>();
2 Predicate<Producto> agotado = p -> {
3     if (p.stock() == 0) agotados.add(p.nombre());
4     return p.stock() == 0;    // ¡un Predicate
    que modifica una lista!
5 };

```

Quien use el predicado no espera que modifique datos. Si se evalúa dos veces, la lista queda duplicada.

✓ PREFIERE

```

1 Predicate<Producto> agotado = p -> p.stock() == 0;
   // solo pregunta
2
3 List<String> agotados = new ArrayList<>();
4 for (Producto p : catalogo) {
5     if (agotado.test(p)) agotados.add(p.nombre());
6     // el efecto, explícito
7 }

```

`Function` y `Predicate` deben ser **puras**; los efectos van en `Consumer` o en código explícito.

Buenas prácticas

#	Práctica	Por qué
1	Prefiere las interfaces de <code>java.util.function</code>	Son conocidas y ya traen composición
2	Anota con <code>@FunctionalInterface</code> las interfaces propias	El compilador protege el contrato
3	Usa las versiones primitivas en código intensivo	Evitas millones de objetos temporales
4	Mantén las lambdas cortas (1–3 líneas)	Si crecen, extrae un método y usa una referencia
5	Escribe funciones puras en <code>Function</code> y <code>Predicate</code>	Son predecibles, testeables y seguras en paralelo
6	No sobrecargues con interfaces funcionales de igual aridad	Evitas llamadas ambiguas para tus usuarios
7	Valida con <code>Objects.requireNonNull</code> en métodos de composición	El error aparece donde está el problema
8	Usa comodines <code>? super</code> / <code>? extends</code> en parámetros funcionales	Tu API acepta más casos válidos
9	Nunca silencies excepciones dentro de una lambda	Conserva la causa al envolverlas
10	Nombra las variables funcionales por su rol: <code>esValido</code> , <code>aEtiqueta</code>	El código se lee como una frase

Bajo el capó: cómo ejecuta Java una lambda

Una clase anónima genera un archivo `.class` propio al compilar. Una lambda no: `javac` traslada su cuerpo a un **método privado sintético** de la clase que la contiene y deja en su lugar una instrucción `invokedynamic`. La primera vez que se ejecuta esa línea, la JVM usa `LambdaMetafactory` para crear en memoria una clase que implementa la interfaz funcional y delega en ese método. Puedes comprobarlo con reflexión:

LISTADO 20 Los métodos sintéticos que genera javac

Main.java

JAVA

```

1  import java.lang.reflect.Method;
2  import java.util.*;
3  import java.util.function.*;
4
5  public class Main {
6      public static void main(String[] args) {
7          Supplier<String> saludo = () -> "hola";
8          Function<Integer, Integer> doble = x -> x * 2;
9          Supplier<String> vacio = String::new;           // referencia: no genera método
10         List<String> sinteticos = new ArrayList<>();
11         for (Method m : Main.class.getDeclaredMethods()) if (m.isSynthetic()) sinteticos.add(m.getName());
12         Collections.sort(sinteticos);
13
14         System.out.println("Métodos sintéticos en Main: " + sinteticos);
15         System.out.println("¿Clase anónima? " + saludo.getClass().isAnonymousClass());
16         System.out.println(saludo.get() + " " + doble.apply(21) + " [" + vacio.get() + "]");
17     }
18 }
```

>_ SALIDA EN CONSOLA

```

$ java Main
Métodos sintéticos en Main: [lambda$main$0, lambda$main$1]
¿Clase anónima? false
hola 42 []
```

☆ NOTA TÉCNICA

Por eso en las trazas de error verás nombres como `lambda$main$0`: es el método sintético que contiene el cuerpo de tu lambda. Y es otra razón para extraer las lambdas largas a métodos con nombre: la traza mostrará

`calcularDescuento` en lugar de `lambda$procesar$3`.

EN RESUMEN · PARTE III

- Crea interfaces propias cuando el concepto tiene nombre en tu dominio, necesitas más de dos parámetros o excepciones verificadas; agrégalas composición con métodos `default`.
- Para excepciones verificadas, define una interfaz con `throws` y un adaptador que conserve la causa.
- Estrategia, comando, observador y decorador se expresan con lambdas en pocas líneas.
- Evita sobrecargas ambiguas, métodos funcionales genéricos y efectos secundarios ocultos.

13 Proyecto integrador: motor de promociones

Laboratorio 4 · Motor de promociones del catálogo

INTEGRADOR · 90 MIN

Objetivo: modelar reglas de negocio configurables usando solo interfaces funcionales. **Secciones:** 3 · 6 · 7 · 8 · 11

La tienda quiere definir promociones sin tocar el código que calcula precios. Cada promoción tiene un nombre, un **criterio** (a qué productos aplica) y un **ajuste** (cómo cambia el precio). Para cada producto disponible, el sistema debe elegir la promoción más conveniente para el cliente.

1 Modela

`record Promocion(String nombre, Predicate<Producto> aplicaA, DoubleUnaryOperator ajuste)` con un método `precioCon(Producto p)`: si aplica, ajusta el precio; si no, lo devuelve igual.

2 Crea dos fábricas de ajustes que devuelvan `DoubleUnaryOperator`: `porcentaje(double pct)` y `montoFijo(double monto)` (sin bajar de cero).

3 Define tres promociones: 10 % en tecnología, bono de \$50.000 para productos de \$200.000 o más, y 30 % de liquidación para productos con más de 100 unidades.

4 Usa un `Supplier<Promocion>` para la opción "Sin promoción", con ajuste `DoubleUnaryOperator.identity()`.

5 Elige la mejor con

`BinaryOperator.minBy(Comparator.comparingDouble(promo -> promo.precioCon(p)))`, reduciendo la lista con un ciclo.

6 Imprime cada fila con un `BiConsumer<Producto, Promocion>` y acumula el ahorro total con una `ToDoubleBiFunction`.

> SALIDA EN CONSOLA

```
$ java Main
Portátil          Tecno 10 %    $ 3.200.000 → $ 2.880.000
Mouse             Tecno 10 %    $   85.000 → $   76.500
Audífonos         Bono $50.000 $ 240.000 → $ 190.000
Lápiz             Liquidación  $   2.500 → $   1.750
Silla ergonómica Bono $50.000 $ 890.000 → $ 840.000
Ahorro total para el cliente: $429.250
```

Observa que el código que *calcula* no conoce ninguna promoción concreta: agregar una nueva es agregar un elemento a la lista. Ese es el objetivo de diseñar con interfaces funcionales.

A Soluciones de referencia

Todas compilan y producen **exactamente** la salida mostrada en cada laboratorio.

Laboratorio 1 · Generador de slugs

Main.java

JAVA

```

1  import java.text.Normalizer;
2  import java.util.List;
3  import java.util.function.Function;
4  import java.util.function.UnaryOperator;
5
6  public class Main {
7      static final UnaryOperator<String> RECORTAR = String::strip;
8      static final UnaryOperator<String> MINUSCULAS = String::toLowerCase;
9      static final UnaryOperator<String> SIN_TILDES =
10         s -> Normalizer.normalize(s, Normalizer.Form.NFD).replaceAll("\\p{M}", "");
11     static final UnaryOperator<String> SOLO_VALIDOS = s -> s.replaceAll("[^a-z0-9\\s-]", "");
12     static final UnaryOperator<String> GUIONES = s -> s.strip().replaceAll("[\\s-]+", "-");
13
14     static Function<String, String> tuberia(List<? extends Function<String, String>> pasos) {
15         Function<String, String> resultado = Function.identity();
16         for (Function<String, String> paso : pasos) resultado = resultado.andThen(paso);
17         return resultado;
18     }
19
20     public static void main(String[] args) {
21         Function<String, String> slug =
22             tuberia(List.of(RECORTAR, MINUSCULAS, SIN_TILDES, SOLO_VALIDOS, GUIONES));
23         for (String nombre : List.of(" Silla Ergonómica ", "Audífonos Bluetooth 5.3",
24             "¡Oferta! Portátil i7 -- 16GB")) {
25             System.out.printf("%-32s -> %s%n", "\"" + nombre + "\"", slug.apply(nombre));
26         }
27     }
28 }

```

Laboratorio 2 · Motor de validación

Main.java

JAVA

```

1  import java.util.ArrayList;
2  import java.util.List;
3  import java.util.function.*;
4
5  @FunctionalInterface
6  interface Regla<T> {
7      List<String> validar(T valor);
8
9      default Regla<T> y(Regla<? super T> otra) {
10         return valor -> {
11             List<String> errores = new ArrayList<>(validar(valor));
12             errores.addAll(otra.validar(valor));
13             return errores;
14         };
15     }
16
17     static <T> Regla<T> exigir(Predicate<? super T> condicion, String mensaje) {
18         return valor -> condicion.test(valor) ? List.of() : List.of(mensaje);
19     }
20
21     static <T, C> Regla<T> campo(Function<? super T, ? extends C> extractor, Regla<? super C> regla) {
22         return valor -> regla.validar(extractor.apply(valor));
23     }
24 }
25
26 record Registro(String usuario, String correo, int edad) { }
27
28 public class Main {

```

```

29     public static void main(String[] args) {
30         Regla<String> usuarioTexto = Regla.<String>exigir(s -> !s.isBlank(), "usuario: obligatorio")
31             .y(Regla.exigir(s -> s.length() >= 4, "usuario: mínimo 4 caracteres"));
32         Regla<Registro> usuarioValido = Regla.campo(Registro::usuario, usuarioTexto);
33         Regla<Registro> correoValido = Regla.exigir(
34             r -> r.correo().matches("[^@\\s]+@[^@\\s]+\\.\\.[a-z]{2,}"), "correo: formato inválido");
35         Regla<Registro> edadValida = Regla.exigir(r -> r.edad() >= 14, "edad: mínimo 14 años");
36         Regla<Registro> todas = usuarioValido.y(correoValido).y(edadValida);
37
38         List<Registro> registros = List.of(new Registro("camila_r", "camila@correo.co", 19),
39             new Registro("", "sin-arroba", 12), new Registro("ana", "ana@correo", 15));
40         for (Registro r : registros) {
41             List<String> errores = todas.validar(r);
42             String usuario = r.usuario().isEmpty() ? "(vacío)" : r.usuario();
43             System.out.printf("%-9s %-10s %s%n", errores.isEmpty() ? "VÁLIDO" : "INVÁLIDO", usuario, errores);
44         }
45     }
46 }

```

Laboratorio 3 · Calculadora con estrategias

Main.java

JAVA

```

1  import java.util.LinkedHashMap;
2  import java.util.List;
3  import java.util.Map;
4  import java.util.function.DoubleBinaryOperator;
5
6  public class Main {
7      static final Map<String, DoubleBinaryOperator> OPERACIONES = new LinkedHashMap<>();
8      static {
9          OPERACIONES.put("+", Double::sum);
10         OPERACIONES.put("-", (a, b) -> a - b);
11         OPERACIONES.put("*", (a, b) -> a * b);
12         OPERACIONES.put("/", (a, b) -> {
13             if (b == 0) throw new ArithmeticException("división por cero");
14             return a / b;
15         });
16         OPERACIONES.put("^", Math::pow);
17         OPERACIONES.put("max", Math::max);
18     }
19
20     static double evaluar(String expresion) {
21         String[] partes = expresion.strip().split("\\s+");
22         if (partes.length != 3) throw new IllegalArgumentException("formato esperado: a op b");
23         DoubleBinaryOperator operacion = OPERACIONES.get(partes[1]);
24         if (operacion == null) throw new IllegalArgumentException("operador desconocido: " + partes[1]);
25         return operacion.applyAsDouble(Double.parseDouble(partes[0]), Double.parseDouble(partes[2]));
26     }
27
28     public static void main(String[] args) {
29         for (String e : List.of("12 + 30", "2 ^ 10", "7 / 2", "9 max 4", "5 / 0", "3 % 2")) {
30             try {
31                 System.out.printf("%-8s = %s%n", e, evaluar(e));
32             } catch (RuntimeException ex) {
33                 System.out.printf("%-8s → error: %s%n", e, ex.getMessage());
34             }
35         }
36         System.out.println("Operadores disponibles: " + OPERACIONES.keySet());
37     }
38 }

```

Laboratorio 4 · Motor de promociones

Main.java

JAVA

```
1 import java.util.*;
2 import java.util.function.*;
3
4 record Promocion(String nombre, Predicate<Producto> aplicaA, DoubleUnaryOperator ajuste) {
5     double precioCon(Producto p) {
6         return aplicaA.test(p) ? ajuste.applyAsDouble(p.precio()) : p.precio();
7     }
8 }
9
10 public class Main {
11     static DoubleUnaryOperator porcentaje(double pct) { return precio -> precio * (1 - pct / 100); }
12     static DoubleUnaryOperator montoFijo(double monto) { return precio -> Math.max(0, precio - monto); }
13
14     public static void main(String[] args) {
15         List<Promocion> promociones = List.of(
16             new Promocion("Tecno 10 %", p -> p.categoria().equals("tecnología"), porcentaje(10)),
17             new Promocion("Bono $50.000", p -> p.precio() >= 200_000, montoFijo(50_000)),
18             new Promocion("Liquidación", p -> p.stock() > 100, porcentaje(30));
19
20         Supplier<Promocion> sinPromocion =
21             () -> new Promocion("Sin promoción", p -> true, DoubleUnaryOperator.identity());
22         BiConsumer<Producto, Promocion> imprimir = (p, promo) -> System.out.printf(
23             "%-17s %-14s $%,11.0f → $%,11.0f%n", p.nombre(), promo.nombre(), p.precio(),
24             promo.precioCon(p));
25         ToDoubleBiFunction<Producto, Promocion> ahorro = (p, promo) -> p.precio() - promo.precioCon(p);
26
27         double ahorroTotal = 0;
28         for (Producto p : Catalogo.muestra()) {
29             if (!p.disponible()) continue;
30             BinaryOperator<Promocion> masConveniente =
31                 BinaryOperator.minBy(Comparator.comparingDouble(promo -> promo.precioCon(p)));
32             Promocion mejor = sinPromocion.get();
33             for (Promocion promo : promociones) mejor = masConveniente.apply(mejor, promo);
34             imprimir.accept(p, mejor);
35             ahorroTotal += ahorro.applyAsDouble(p, mejor);
36         }
37         System.out.printf("Ahorro total para el cliente: $%,.0f%n", ahorroTotal);
38     }
39 }
```

B

Catálogo completo de

java.util.function

Las 43 interfaces del Listado 9, organizadas por familia y por tipo primitivo.

Familia	Genéricas	int	Long	double	Total
Function transforma	Function	IntFunction	LongFunction	DoubleFunction	17
	BiFunction	ToIntFunction	ToLongFunction	ToDoubleFunction	
		ToIntBiFunction	ToLongBiFunction	ToDoubleBiFunction	
		IntToLongFunction	LongToIntFunction	DoubleToIntFunction	
		IntToDoubleFunction	LongToDoubleFunction	DoubleToLongFunction	
Operator mismo tipo	UnaryOperator	IntUnaryOperator	LongUnaryOperator	DoubleUnaryOperator	8
	BinaryOperator	IntBinaryOperator	LongBinaryOperator	DoubleBinaryOperator	
Predicate pregunta	Predicate	IntPredicate	LongPredicate	DoublePredicate	5
	BiPredicate				

Familia	Genéricas	int	long	double	Total
Consumer actúa	<code>Consumer</code> <code>BiConsumer</code>	<code>IntConsumer</code> <code>ObjIntConsumer</code>	<code>LongConsumer</code> <code>ObjLongConsumer</code>	<code>DoubleConsumer</code> <code>ObjDoubleConsumer</code>	8
Supplier produce	<code>Supplier</code>	<code>IntSupplier</code>	<code>LongSupplier</code>	<code>DoubleSupplier</code>	4 + <code>BooleanSupplier</code> = 5
Total del paquete					43

C Referencia rápida y autoevaluación

Todo lo esencial en [una página](#), y diez preguntas para comprobar tu dominio.

DEFINIR Y ANOTAR

```
@FunctionalInterface
interface Regla<T> {
    boolean cumple(T t);           // único
    abstracto
    default Regla<T> y(Regla<? super T> o) {
        return t -> cumple(t) && o.cumple(t);
    }
    static <T> Regla<T> siempre() { return t ->
true; }
}
// default, static, private y métodos de Object
// NO cuentan como abstractos
```

IMPLEMENTAR

```
x -> x * 2                (a, b) -> a + b
() -> "valor"             (var s) -> s.strip()
s -> { var t = s.strip(); return t; }

Clase::estatico           Integer::parseInt
objeto::metodo            System.out::println
Clase::instancia         String::toUpperCase
Clase::new                ArrayList::new, int[]::new
```

LAS CUATRO FAMILIAS

```
Function<T,R>   R apply(T)      transforma
Predicate<T>    boolean test(T) pregunta
Consumer<T>     void accept(T)  actúa
Supplier<T>     T get()         produce
Bi...  → 2 parámetros   ...Operator → mismo tipo
Int...  → entrada int   ToInt...  → salida int
```

COMPONER

```
f.andThen(g) // g(f(x))
f.compose(g) // f(g(x))
Function.identity()
p.and(q) p.or(q) p.negate() Predicate.not(p)
c1.andThen(c2)
BinaryOperator.minBy(cmp) maxBy(cmp)
Comparator.comparing(k).thenComparing(k2)
```

1 ¿Cuál de estas es una interfaz funcional?

- a) `interface A { void x(); void y(); }`
- b) `interface B { void x(); boolean equals(Object o); }`
- c) `interface C { default void x() { } }`
- d) `abstract class D { abstract void x(); }`

2 ¿Qué ocurre con `var f = x -> x + 1; ?`

- a) Compila como `Function`
- b) Error: la lambda necesita tipo destino
- c) Compila como `IntUnaryOperator`
- d) Error de sintaxis en la flecha

3 Dentro del cuerpo de una lambda, `this` se refiere a...

- a) La propia lambda
- b) La instancia que la contiene
- c) `null`
- d) La interfaz funcional

4 `String::length` usada como `Function<String, Integer>` es una referencia a...

- a) Método estático
- b) Método de un objeto concreto
- c) Método de instancia de un objeto arbitrario
- d) Constructor

5 ¿Qué calcula `f.andThen(g).apply(x)` ?

- a) `f(g(x))`
- b) `g(f(x))`
- c) `f(x) + g(x)`
- d) Depende de los tipos

6 ¿Qué interfaz convierte un `Producto` en `double` sin autoboxing?

- a) `Function<Producto, Double>`
- b) `ToDoubleFunction<Producto>`
- c) `DoubleFunction<Producto>`
- d) `DoubleSupplier`

7 Una lambda hace `contador++` sobre una variable **local**. ¿Qué pasa?

- a) Compila y funciona
- b) Error: debe ser efectivamente final
- c) Excepción en ejecución
- d) Solo compila si es `static`

8 `Function<String, URI> f = s -> new URI(s);`

- a) Compila sin problemas
- b) Error: excepción verificada no reportada
- c) Compila con una advertencia
- d) Falla solo en ejecución

9 ¿Para qué es especialmente útil `Supplier<T>` ?

- a) Evaluación diferida de valores
- b) Filtrar colecciones
- c) Recibir dos parámetros
- d) Comparar objetos

10 Existen `ejecutar(Supplier<String>)` y `ejecutar(Callable<String>)` . ¿Qué pasa con `ejecutar(() -> "x")` ?

- a) Llama a la versión `Supplier`
- b) Llama a la versión `Callable`
- c) Error: llamada ambigua
- d) Error en ejecución

CLAVE DE RESPUESTAS

1 b (§2) · 2 b (§3) · 3 b (§3) · 4 c (§4) · 5 b (§7) · 6 b (§8) · 7 b (§5) · 8 b (§10) · 9 a (§11) · 10 c (§12). Con 8 o más aciertos dominas el tema; si no, repasa las secciones indicadas.

RETOS PARA PROFUNDIZAR

1. Escribe `TriFunction.curry()` , que convierta una `TriFunction<A,B,C,R>` en `Function<A, Function<B, Function<C, R>>>` . 2. Agrega a `Regla<T>` los métodos `o(...)` y `cuando(Predicate<T> condicion)` , que aplique la regla solo si se cumple la condición. 3. Crea `memorizar` segura para hilos con `ConcurrentHashMap` y explica qué pasa si la función es recursiva. 4. Implementa un `Reintentar.de(Supplier<T>, int intentos)` que vuelva a ejecutar el proveedor si lanza una excepción.

Glosario

Interfaz funcional: interfaz con exactamente un método abstracto.

Descriptor funcional: firma del SAM (parámetros, retorno y excepciones).

Lambda: implementación anónima del SAM:
`(params) -> cuerpo` .

Efectivamente final: variable local que nunca se reasigna; es la única capturable.

Composición: construir una función nueva a partir de otras (`andThen` , `and` ...).

SAM: *Single Abstract Method*, el método funcional de la interfaz.

Tipo destino: interfaz funcional esperada en el contexto de una lambda.

Referencia a método: forma abreviada de una lambda que solo llama a un método: `Clase::metodo` .

Closure: función que conserva acceso a variables de su entorno de creación.

Función pura: sin efectos secundarios; el mismo dato de entrada da siempre el mismo resultado.

Autoboxing: conversión automática de primitivo a objeto envoltorio (`int` → `Integer`).

Currificación: transformar una función de n parámetros en una cadena de funciones de uno.

PECS: *Producer Extends, Consumer Super*, la regla para usar comodines en genéricos.

Función de orden superior: recibe o devuelve funciones.

Memoización: guardar resultados ya calculados para no repetir el trabajo.

`invokedynamic` : instrucción de la JVM que enlaza una lambda con su implementación en ejecución.

Lista de verificación de dominio

✓	Puedo...	Nivel	\$
☐	Decidir si una interfaz es funcional y explicar por qué	COMPRENDER	2
☐	Predecir el tipo destino de una lambda y los errores cuando falta	ANALIZAR	3
☐	Convertir lambdas en los cuatro tipos de referencia a métodos	APLICAR	4
☐	Explicar la regla de captura y por qué existe	COMPRENDER	5
☐	Elegir la interfaz estándar adecuada, incluidas las primitivas	APLICAR	6 · 8
☐	Componer funciones, predicados, consumidores y comparadores	APLICAR	7
☐	Diseñar una interfaz propia con composición, fábricas y comodines	CREAR	9
☐	Adaptar código con excepciones verificadas sin perder la causa	APLICAR	10
☐	Expresar estrategia, comando, observador y decorador con lambdas	CREAR	11
☐	Revisar código ajeno y señalar sobrecargas ambiguas y efectos ocultos	EVALUAR	12

En una frase: una interfaz funcional es el **contrato mínimo** —un solo método— que permite tratar el comportamiento como un dato; las lambdas y las referencias a métodos son la forma de escribirlo, y la composición es la forma de hacerlo crecer sin escribir clases.

Todos los listados fueron compilados y ejecutados con Java 21 LTS (configuración regional es-CO); los mensajes de error se obtuvieron del compilador `javac`.